

ASSIGNMENT-16.1

Name:G.Kalyan

Ht.no:2303a51853

Batch:13

Task 1 – Student Information System Schema

Task:

Design a database schema for a Student Information System and generate queries using AI.

Instructions:

- Tables: Students, Courses, Enrollments.
- Define primary keys, foreign keys, and relationships.
- Generate queries:
 - o Insert a new student record.
 - o Fetch all courses enrolled by a student.
 - o Count number of students in each course.

Expected Output:

- SQL CREATE TABLE statements and queries for student– course relationships.

Definitions:

Primary Key: A Primary Key uniquely identifies each record in a table.

Foreign Key: A Foreign Key is used to link tables together.

3. Relationships Between Tables

Type of Relationship:

Many-to-Many (M:N) between Students and Courses

- One student can enroll in many courses
- One course can have many students

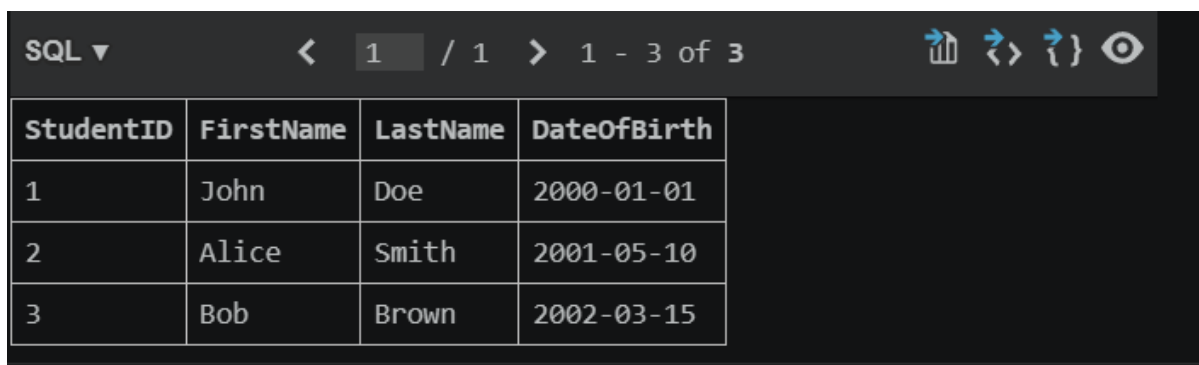
- **Generate queries:**

- o Insert a new student record.
- o Fetch all courses enrolled by a student.
- o Count number of students in each course.

Query:

```
47 -- Insert Enrollments
48 INSERT INTO Enrollments (StudentID, CourseID)
49 VALUES (1, 1);
50
51 INSERT INTO Enrollments (StudentID, CourseID)
52 VALUES (1, 2);
53
54 INSERT INTO Enrollments (StudentID, CourseID)
55 VALUES (2, 1);
56
57 -- =====
58 -- QUERIES
59 -- =====
60
61 -- 1. Insert a New Student Record
62 INSERT INTO Students (FirstName, LastName, DateOfBirth)
63 VALUES ('Bob', 'Brown', '2002-03-15');
64
65 -- 2. Fetch All Courses Enrolled by a Student (StudentID = 1)
66 SELECT c.CourseName, c.Credits
67 FROM Enrollments e
68 JOIN Courses c ON e.CourseID = c.CourseID
69 WHERE e.StudentID = 1;
70
71 -- 3. Count Number of Students in Each Course
72 SELECT c.CourseName, COUNT(e.StudentID) AS NumberOfStudents
73 FROM Courses c
74 LEFT JOIN Enrollments e ON c.CourseID = e.CourseID
75 GROUP BY c.CourseID;
76 Select * from Students;
```

Output:



SQL ▾ < 1 / 1 > 1 - 3 of 3

StudentID	FirstName	LastName	DateOfBirth
1	John	Doe	2000-01-01
2	Alice	Smith	2001-05-10
3	Bob	Brown	2002-03-15

Justification: The relationship between the *Students* and *Courses* tables is a many-to-many relationship, because a single student can enroll in multiple courses, and each course can have multiple students. This relationship cannot be represented directly in a single table, so it is implemented using the Enrollments table, which acts as a junction table.

SQL ▾		<	1	/ 1	>	1 - 2 of 2	   
CourseName	Credits						
Database Systems	4						
Operating Systems	3						

SQL ▾		<	1	/ 1	>	1 - 2 of 2	   
CourseName	NumberOfStudents						
Database Systems	2						
Operating Systems	1						

SQL ▾		<	1	/ 1	>	1 - 3 of 3	   
StudentID	FirstName	LastName	DateOfBirth				
1	John	Doe	2000-01-01				
2	Alice	Smith	2001-05-10				
3	Bob	Brown	2002-03-15				

Task 2 – Hospital Management Database

Task:

Create schema and queries for a Hospital Management System.

Instructions:

- Tables: Doctors, Patients, Appointments.
- Use AI to define constraints (unique IDs, valid dates).
- Generate queries:
 - o List all appointments for a specific doctor.
 - o Retrieve patient history by patient ID.
 - o Count total patients treated by each doctor.

Expected Output:

- Normalized schema and SQL queries with joins.

Queries

```
26 -- Insert Doctors
27 INSERT INTO Doctors (Name, Specialization) VALUES ('Dr. Smith', 'Cardiology');
28 INSERT INTO Doctors (Name, Specialization) VALUES ('Dr. John', 'Neurology');
29 INSERT INTO Doctors (Name, Specialization) VALUES ('Dr. Emily', 'Pediatrics');
30
31 -- Insert Patients
32 INSERT INTO Patients (Name, DateOfBirth) VALUES ('Alice', '1995-06-15');
33 INSERT INTO Patients (Name, DateOfBirth) VALUES ('Bob', '1990-03-20');
34 INSERT INTO Patients (Name, DateOfBirth) VALUES ('Charlie', '1985-12-05');
35
36 -- Insert Appointments
37 INSERT INTO Appointments (DoctorID, PatientID, AppointmentDate, Diagnosis)
38 VALUES (1, 1, '2025-03-01', 'Heart Checkup');
39
40 INSERT INTO Appointments (DoctorID, PatientID, AppointmentDate, Diagnosis)
41 VALUES (1, 2, '2025-03-05', 'BP Issue');
42 -- Query 1. List all appointments for a specific doctor
43 INSERT INTO Appointments (DoctorID, PatientID, AppointmentDate, Diagnosis)
44 VALUES (2, 1, '2025-03-10', 'Migraine');
45 SELECT d.Name AS DoctorName, p.Name AS PatientName, a.AppointmentDate, a.Diagnosis
46 FROM Appointments a
47 JOIN Doctors d ON a.DoctorID = d.DoctorID
48 JOIN Patients p ON a.PatientID = p.PatientID
49 WHERE d.DoctorID = 1;
50 -- Query 2. Retrieve patient history by patient ID
51 SELECT p.Name AS PatientName, d.Name AS DoctorName, a.AppointmentDate, a.Diagnosis
52 FROM Appointments a
53 JOIN Patients p ON a.PatientID = p.PatientID
54 JOIN Doctors d ON a.DoctorID = d.DoctorID
55 WHERE p.PatientID = 1;
56 -- Query 3. Count total patients treated by each doctor
57 SELECT d.Name AS DoctorName, COUNT(DISTINCT a.PatientID) AS TotalPatients
58 FROM Doctors d
59 LEFT JOIN Appointments a ON d.DoctorID = a.DoctorID
60 GROUP BY d.DoctorID;
```

Output:

DoctorName	PatientName	AppointmentDate	Diagnosis
Dr. Smith	Alice	2025-03-01	Heart Checkup
Dr. Smith	Bob	2025-03-05	BP Issue

SQL ▾ < 1 / 1 > 1 - 2 of 2

PatientName	DoctorName	AppointmentDate	Diagnosis
Alice	Dr. Smith	2025-03-01	Heart Checkup
Alice	Dr. John	2025-03-10	Migraine

SQL ▾ < 1 / 1 > 1 - 3 of 3

DoctorName	TotalPatients
Dr. Smith	2
Dr. John	1
Dr. Emily	0

Ln 59, Col 52 Spaces: 4 UTF-8 CRLF {} SQL SQLite: :memory: Go Live

Justification: The Appointments table links doctors and patients via foreign key. Constraints like NOT NULL, UNIQUE, and valid date checks ensure data integrity, enabling efficient storage, retrieval, and querying of hospital data.

Task 3 – Library Database

Task:

Design schema for a Library Management System.

Instructions:

- Tables: Books, Members, Loans.
- Use AI to suggest indexing strategy for faster queries.
- Generate queries:
 - o Retrieve all books currently issued.
 - o Find overdue books (loan date > 30 days).
 - o Count number of books loaned by each member.

Expected Output:

- Schema + SQL queries demonstrating joins and conditions.

Queries

```
31 -- Books
32 INSERT INTO Books (Title, Author, AvailableCopies) VALUES ('DBMS', 'Author A', 5);
33 INSERT INTO Books (Title, Author, AvailableCopies) VALUES ('OS', 'Author B', 3);
34
35 -- Members
36 INSERT INTO Members (Name, JoinDate) VALUES ('Alice', '2024-01-01');
37 INSERT INTO Members (Name, JoinDate) VALUES ('Bob', '2024-02-01');
38
39 -- Loans
40 INSERT INTO Loans (BookID, MemberID, LoanDate, ReturnDate)
41 VALUES (1, 1, '2025-02-01', NULL); -- currently issued
42
43 INSERT INTO Loans (BookID, MemberID, LoanDate, ReturnDate)
44 VALUES (2, 2, '2025-01-01', NULL); -- overdue
45
46 INSERT INTO Loans (BookID, MemberID, LoanDate, ReturnDate)
47 VALUES (1, 2, '2025-03-01', '2025-03-10'); -- returned
48 -- Query 1: Retrieve all books currently issued
49 SELECT b.Title, m.Name AS MemberName, l.LoanDate
50 FROM Loans l
51 JOIN Books b ON l.BookID = b.BookID
52 JOIN Members m ON l.MemberID = m.MemberID
53 WHERE l.ReturnDate IS NULL;
54 -- Query 2: Find overdue books (loan date > 30 days)
55 SELECT b.Title, m.Name AS MemberName, l.LoanDate
56 FROM Loans l
57 JOIN Books b ON l.BookID = b.BookID
58 JOIN Members m ON l.MemberID = m.MemberID
59 WHERE l.ReturnDate IS NULL
60 AND DATE(l.LoanDate) <= DATE('now', '-30 days');
61 -- Query 3: Count number of books loaned by each member
62 SELECT m.Name AS MemberName, COUNT(l.BookID) AS TotalBooksLoaned
63 FROM Members m
64 LEFT JOIN Loans l ON m.MemberID = l.MemberID
65 GROUP BY m.MemberID;
```

Output

Title	MemberName	LoanDate
DBMS	Alice	2025-02-01
OS	Bob	2025-01-01

SQL ▾ < 1 / 1 > 1 - 2 of 2

Title	MemberName	LoanDate
DBMS	Alice	2025-02-01
OS	Bob	2025-01-01

SQL ▾ < 1 / 1 > 1 - 2 of 2

MemberName	TotalBooksLoaned
Alice	1
Bob	2

Justification: The library database is designed using three normalized tables: Books, Members, and Loans. Each table has a primary key to uniquely identify records, while the Loans table uses foreign keys to establish relationships between books and members.

Task 4 – Real-Time Application: Online Shopping Database

Scenario:

Design a database for an E-commerce platform.

Requirements:

- Tables: Users, Products, Orders, OrderDetails.
- Generate queries to:
 - o Retrieve all orders by a user.
 - o Find the most purchased product.
 - o Calculate total revenue in a given month.
- AI should also suggest normalization improvements and query optimization.

Expected Output:

- Complete schema with relationships + SQL queries for analytics.

Queries

```
32 -- Users
33 INSERT INTO Users (Name, Email) VALUES ('Alice', 'alice@gmail.com');
34 INSERT INTO Users (Name, Email) VALUES ('Bob', 'bob@gmail.com');
35
36 -- Products
37 INSERT INTO Products (ProductName, Price) VALUES ('Laptop', 50000);
38 INSERT INTO Products (ProductName, Price) VALUES ('Phone', 20000);
39
40 -- Orders
41 INSERT INTO Orders (UserID, OrderDate) VALUES (1, '2025-03-01');
42 INSERT INTO Orders (UserID, OrderDate) VALUES (1, '2025-03-10');
43 INSERT INTO Orders (UserID, OrderDate) VALUES (2, '2025-03-15');
44
45 -- OrderDetails
46 INSERT INTO OrderDetails (OrderID, ProductID, Quantity) VALUES (1, 1, 1);
47 INSERT INTO OrderDetails (OrderID, ProductID, Quantity) VALUES (1, 2, 2);
48 INSERT INTO OrderDetails (OrderID, ProductID, Quantity) VALUES (2, 2, 1);
49 INSERT INTO OrderDetails (OrderID, ProductID, Quantity) VALUES (3, 1, 1);
50
51 -- Query 1: Retrieve all orders by a user
52 SELECT u.Name, o.OrderID, o.OrderDate
53 FROM Orders o
54 JOIN Users u ON o.UserID = u.UserID
55 WHERE u.UserID = 1;
56
57 -- Query 2: Find the most purchased product
58 SELECT p.ProductName, SUM(od.Quantity) AS TotalSold
59 FROM OrderDetails od
60 JOIN Products p ON od.ProductID = p.ProductID
61 GROUP BY p.ProductID
62 ORDER BY TotalSold DESC
63 LIMIT 1;
64
65 -- Query 3: Calculate total revenue in a given month
66 SELECT SUM(p.Price * od.Quantity) AS TotalRevenue
67 FROM OrderDetails od
68 JOIN Products p ON od.ProductID = p.ProductID
69 JOIN Orders o ON od.OrderID = o.OrderID
70 WHERE strftime('%Y-%m', o.OrderDate) = '2025-03';
```

Output

Name	OrderID	OrderDate
Alice	1	2025-03-01
Alice	2	2025-03-10

SQL ▾ < 1 / 1 > 1 - 1 of 1

ProductName	TotalSold
Phone	3

SQL ▾ < 1 / 1 > 1 - 1 of 1

TotalRevenue
160000.0

Ln 67, Col 50 (2292 selected) Spaces: 4 UTF-8 CRLF { } SQL SQLite: :memory: Go Live

Justification: The database is structured in a normalized form to eliminate redundancy and ensure data integrity. The Users table stores customer details, while the Products table maintains product information independently. Orders are separated into a different table to record each transaction, and OrderDetails acts as a junction table to handle the many-to-many relationship between orders and products.

Task 5 – University Examination Database

Design a database schema for a University Examination System and generate SQL queries using AI.

Instructions:

- Tables: Students, Subjects, Exams, Results.
- Define primary keys, foreign keys, and referential integrity constraints.
- Generate queries:
 - o Insert examination results for a student.
 - o Retrieve subject-wise marks for a student.
 - o Calculate average marks for each subject.

Expected Output:

- Normalized schema and SQL queries involving aggregation and joins.

Queries

```
-- Students
INSERT INTO Students (Name) VALUES ('Alice');
INSERT INTO Students (Name) VALUES ('Bob');

-- Subjects
INSERT INTO Subjects (SubjectName) VALUES ('Math');
INSERT INTO Subjects (SubjectName) VALUES ('Science');

-- Exams
INSERT INTO Exams (SubjectID, ExamDate) VALUES (1, '2025-03-01');
INSERT INTO Exams (SubjectID, ExamDate) VALUES (2, '2025-03-05');

-- Results
INSERT INTO Results (StudentID, ExamID, Marks) VALUES (1, 1, 85);
INSERT INTO Results (StudentID, ExamID, Marks) VALUES (1, 2, 90);
INSERT INTO Results (StudentID, ExamID, Marks) VALUES (2, 1, 75);
-- Query 1: Insert examination result for a student
INSERT INTO Results (StudentID, ExamID, Marks)
VALUES (2, 2, 88);
-- Query 2: Retrieve subject-wise marks for a student
SELECT s.SubjectName, r.Marks
FROM Results r
JOIN Exams e ON r.ExamID = e.ExamID
JOIN Subjects s ON e.SubjectID = s.SubjectID
WHERE r.StudentID = 1;
-- Query 3: Calculate average marks for each subject
SELECT s.SubjectName, AVG(r.Marks) AS AverageMarks
FROM Results r
JOIN Exams e ON r.ExamID = e.ExamID
JOIN Subjects s ON e.SubjectID = s.SubjectID
GROUP BY s.SubjectID;
```


Output:

SubjectName	Marks
Math	85
Science	90

SQL ▾ < 1 / 1 > 1 - 2 of 2

SubjectName	AverageMarks
Math	80.0
Science	89.0

SQL ▾ < 1 / 1 > 1 - 4 of 4

ResultID	StudentID	ExamID	Marks
1	1	1	85
2	1	2	90
3	2	1	75
4	2	2	88

Justification: The database is designed using four normalized tables: Students, Subjects, Exams, and Results. Each table has a primary key to uniquely identify records, while foreign keys establish relationships between them. The Exams table links subjects with exam dates, and the Results table acts as a junction table connecting students and exams, enabling a many-to-many relationship. Referential integrity is maintained through foreign key constraints, ensuring that results cannot exist without valid students and exams.